

Partie 2: Programmation Python

CHAPITRE 2

Les Conditions et les Boucles

Module : Informatique 1 : Algorithmique 1/Python



Pr. Mariya OUAISSA
E-mail : m.ouaissa@uca.ac.ma
Département : Informatique FSSM

Année Universitaire : 2023/2024

Plan

L'indentation

Les structures conditionnelles : **if ...else** / **if ...elif...else**

La boucle **for**

La boucle **while**

Les instructions **break** et **continue**

Notion de bloc d'instructions et d'indentation

- En programmation, il est courant de répéter un certain nombre de choses (avec les boucles) ou de faire des choix (avec les tests)
- Par exemple, imaginez que vous souhaitiez répéter 10 fois trois instructions différentes, les unes à la suite des autres. Regardez l'exemple suivant en pseudocode :

```
instruction_qui_indique_à_Python_de_répéter_10_fois_ce_qui_suit:  
    sous-instruction1  
    sous-instruction2  
    sous-instruction3  
instruction_suivante
```

- La première ligne correspond à une instruction qui va indiquer à Python de répéter 10 fois d'autres instructions (il s'agit d'une boucle, on verra le nom de la commande exacte plus tard). Dans le pseudocode ci-dessus, il y a trois instructions à répéter, nommées sous-instruction1 à sous-instruction3.

Notion de bloc d'instructions et d'indentation

- Notez bien les détails de la syntaxe. La première ligne indique que l'on veut répéter une ou plusieurs instructions, elle se termine par « : ». Ce symbole « : » indique à Python qu'il doit attendre un bloc d'instructions, c'est-à-dire un certain nombre de sous-instructions à répéter.
- Python reconnaît un bloc d'instructions, car il est indenté. L'indentation est le décalage d'un ou plusieurs espaces ou tabulations des instructions ***sous-instruction1*** à sous-instruction3, par rapport à ***instruction qui indique à Python de répéter 10 fois ce qui suit:***.
- Pratiquement, l'indentation en Python doit être homogène (soit des espaces, soit des tabulations, mais pas un mélange des deux). Une indentation avec quatre espaces est le style d'indentation le plus traditionnel et celui qui permet une bonne lisibilité du code.
- Enfin, la ligne ***instruction suivante*** sera exécutée une fois que le bloc d'instructions sera terminé.

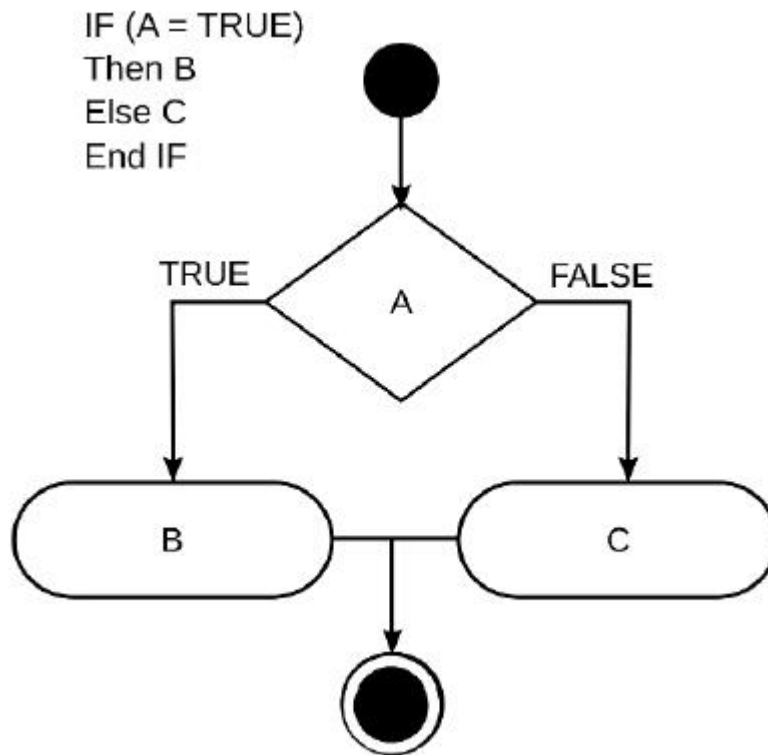
Prendre des décisions en Python



Les conditions

Les instructions conditionnelles

- Un programme doit souvent prendre en compte différents cas et d'adapter à des situations. Il est alors nécessaire d'écrire des conditions.
- Une **condition** est une expression logique dont le résultat est soit « **vrai** » soit « **faux** »



Les instructions conditionnelles- Comparaison

- Une condition est construite à l'aide d'opérateurs de comparaison :

L'opérateur « égale » noté ==

L'opérateur « différent de » noté != ou <>

Les opérateurs « inférieur à » ou « supérieur à » notés < et >

Les opérateurs « inférieur ou égal à » ou « supérieur ou égal à » notés <= et >=

- Lorsque la situation à tester est plus compliquée, il est possible de combiner plusieurs conditions grâce aux opérateurs logiques : « **and** » « **or** » ou « **not** »

Exemple : une condition qui vérifie qu'une distance est inférieure à 40m et qu'un accès wifi est autorisé :

distance < 40 and acces == "autorisé"

Syntaxe python

Une condition doit être suivie par deux points

Les instructions à exécuter si la condition est réalisée sont indentées, c'est-à-dire décalées vers la droite avec une tabulation.

Les Opérateurs Booléens

and

qui vérifie si les deux instructions sont vraies;

- L'opérateur booléen **and** renvoie **True** lorsque les expressions des deux côtés de **and** sont vraies.

2 < 3 and 3 < 4

True

2 > 3 and 3 < 4

False

X	Y	X and Y
True	True	True
False	True	False
True	False	False
False	False	False

Les Opérateurs Booléens

or

qui vérifie si au moins l'une des déclarations est True;

- L'opérateur booléen **or** retourne **True** lorsque au moins une expression de chaque côté de **ou** est vraie. Par exemple:

2 < 3 or 3 < 4

True

2 > 3 or 3 < 4

True

2 > 3 or 3 > 4

False

X	Y	X or Y
True	True	True
False	True	True
True	False	True
False	False	False

Les Opérateurs Booléens

not

ce qui donne le contraire de la déclaration.

- L'opérateur booléen **not** renvoie **True** pour les fausses instructions et **False** pour les vraies instructions. Par exemple:

not False

True

not 2 > 3

True

not 3 < 4

False

not

X	Not X
True	False
False	True

Les instructions conditionnelles - **if**

- Suivant la valeur d'une condition (vraie ou fausse), le programme choisit les actions à réaliser. On parle de structures conditionnelles.
- La structure conditionnelle « **if** » permet d'exécuter un bloc d'instruction lorsqu'une condition est vérifiée.

Exemple : programme qui affiche l'autorisation d'accès d'un membre à un groupe privé.

```
if membre == "oui":  
    print("accès autorisé")
```

Les instructions conditionnelles – if ... else

- La structure conditionnelle « **if...else** » permet d'exécuter un bloc d'instructions lorsqu'une condition est vérifiée et un autre bloc lorsqu'elle ne l'est pas
- Exemple : On affiche maintenant l'interdiction au groupe en cas de réponse fausse :

```
if membre == "oui":  
    print("Accès autorisé")  
else :  
    print("Accès refusé")
```

Example

```
txt = "The best things in life are free!"  
if "free" in txt:  
    print("Yes, 'free' is present.")  
else:  
    print("no, 'free' is not present")
```

Exercices

Exercice 1: Ecrire un programme qui demande un nombre a l'utilisateur, et l'informe ensuite si ce nombre est positif.

Exercice 2: Ecrire un programme demandant à l'utilisateur de donner sa moyenne au bac et affichant s'il est admis.

Ex1

```
1. reponse = int(input ("Entrer un numero!"))  
2. if reponse > 0 :  
3.     print "ton numero est positif"
```

Ex2

```
• note = float(input ("Entrez votre moyenne  
obtenue au bac "))  
• if note >= 10 :  
•     print ("Vous êtes admis")
```

Les instructions conditionnelles – if...elif...else

- La structure conditionnelle « **if...elif...else** » permet de gérer plusieurs conditions. Si une condition n'est pas validée, la suivante est étudiée.
- En Python, « **elif** » est contraction de **else if** qui signifie « *sinon si* »
- Exemple : Programme qui affiche les préférences de confidentialité d'un groupe

```
if statut == "secret":  
    print("Seuls les membres voient le groupe")  
elif statut == "fermé":  
    print("Tout le monde voit le groupe mais pas les publications")  
else:  
    print("Tout le monde voit le groupe et les publications")
```


Exercices

- Exercice 3:
 - Demander l'âge de la personne
 - Si l'âge est supérieur à 60, afficher «c'est une personne âgée»
 - Si l'âge est inférieur à 15, afficher «c'est un enfant»
 - Sinon, afficher «c'est un adulte»

Ex3

```
• age = int(input("âge de la personne ?"))  
• if age > 60:  
    • print "c'est une personne âgée"  
• elif age < 15:  
    • print "c'est un enfant"  
• else:  
    • print "c'est un adulte"
```

Exercices

- Exercice 4: Ecrire un programme demandant à l'utilisateur de donner sa moyenne au bac et affichant s'il est admis, ainsi que sa mention.

[12-14] mention AB

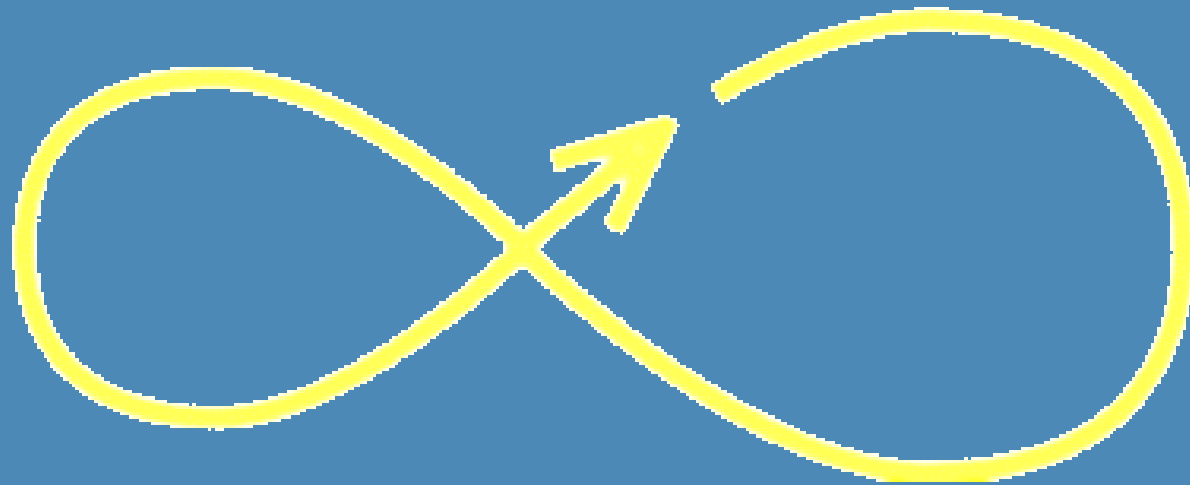
[14-16] mention B

[16 - > 16] mention TB

Ex4

```
• note = float(input ("Entrez votre moyenne  
obtenue au bac "))  
• if note >= 10 :  
• print ("Vous êtes admis")  
  • if note >= 12 and note < 14 :  
    • print ("Mention AB")  
  • if note >= 14 and note < 16 :  
    • print ("Mention B")  
  • if note >= 16 :  
    • print ("Mention TB")  
• else  
• print ("Vous n'êtes pas admis")
```

Les Boucles



Situation

Donner au moins deux versions d'un programme qui permet d'afficher le mot **ANPI** 10 fois,

Version 1

```
1 print('ANPI')
2 print('ANPI')
3 print('ANPI')
4 print('ANPI')
5 print('ANPI')
6 print('ANPI')
7 print('ANPI')
8 print('ANPI')
9 print('ANPI')
10 print('ANPI')
```

Version 2

```
1 for i in range(10):
2     print('ANPI')
```

Les boucles pour permettent de répéter une instruction ou plusieurs un nombre donné de fois.

pour i variant de 1 à n **faire**

instruction 1;
instruction 2;
instruction 3;

.....

Fin Pour

Suite de l'algorithme

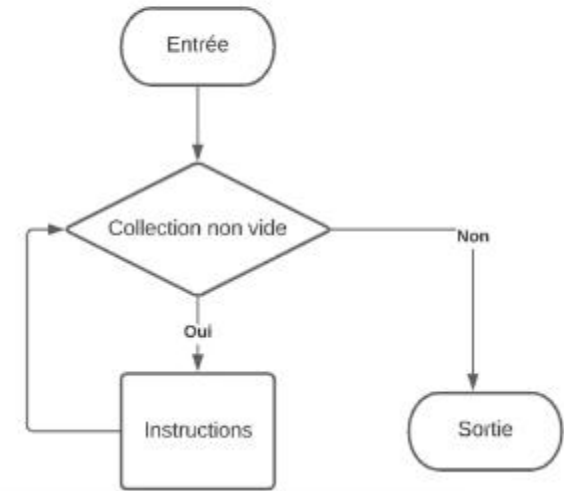


Diagram illustrating the syntax of a `for` loop with annotations:

- `for` and `in`: mots réservés "for" et "in"
- `i`: une variable
- `range(n)`: liste à parcourir, ici $0, 1, 2, \dots, n-1$
- `:`: deux points
- Indented block (`instruction_1`, `instruction_2`, `...`): bloc d'instructions indenté sera répété n fois pour $i = 0$, puis $i = 1 \dots$ jusqu'à $i = n-1$
- Next line (`instructions suivantes`): indentation
- Continuation (`suite du programme`): suite du programme

Les boucles bornées

Une boucle permet de répéter une ou plusieurs instructions.

Lorsque l'on connaît le nombre de répétitions, on utilise une **boucle bornée**.

L'instruction Python correspondant à une boucle bornée est « **for *indice* in range()** » :

- « **in range()** » permet d'énumérer à une boucle de passage dans la boucle bornée
- **L'indice** *i* de la boucle prend les valeurs entières de 0 à *n*-1, donc *n* valeurs. Il est possible de préciser la première et la dernière valeur de l'indice dans la boucle en définissant des bornes. Lorsque l'indice ne doit pas varier de 1 en 1, on peut également modifier le nombre de pas qui sépare un indice du suivant.

La boucle for

CE QU'IL FAUT SAVOIR

syntaxe

```
for variable in range (...) :  
    instruction(s)
```

les différentes façons d'utiliser « range () »

	exemple	
<pre>for k in range (n) :</pre> pour un entier k allant de 0 à $n - 1$	<pre>for k in range (4) :</pre> <pre> print(k)</pre>	<pre>>>> 0 1 2 3</pre> dans la console
<pre>for k in range (n,m) :</pre> pour un entier k allant de n à $m - 1$	<pre>for k in range (5,8) :</pre> <pre> print(k)</pre>	<pre>>>> 5 6 7</pre> dans la console
<pre>for k in range (n,m,p) :</pre> pour un entier k allant de n à $m - 1$ avec un pas de p	<pre>for k in range (2,18,5) :</pre> <pre> print(k)</pre>	<pre>>>> 2 7 12 17</pre> dans la console

La boucle for

CE QU'IL FAUT SAVOIR

syntaxe

```
for variable in range (...):  
    instruction(s)
```

ATTENTION !!

les différentes façons d'utiliser range ()

exemple		
<pre>for k in range(n):</pre> pour un entier k allant de 0 à $n - 1$	<pre>for k in range(4): print(k)</pre>	<pre>>>> 0 1 2 3</pre> dans la console
<pre>for k in range(n,m):</pre> pour un entier k allant de n à $m - 1$	<pre>for k in range(5,8): print(k)</pre>	<pre>>>> 5 6 7</pre> dans la console
<pre>for k in range(n,m,p):</pre> pour un entier k allant de n à $m - 1$ avec un pas de p	<pre>for k in range(2,18,5): print(k)</pre>	<pre>>>> 2 7 12 17</pre> dans la console

La boucle for

Traduire en Python :

Pour un entier k allant de 0 à 17

La boucle for

Traduire en Python :

Pour un entier k allant de 0 à 17

réponse

```
| for k in range(18):
```

valeurs prises par k

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

La boucle for

Traduire en Python :

Pour un entier k allant de 1 à 17

La boucle for

Traduire en Python :

Pour un entier k allant de 1 à 17

réponse

```
for k in range(1,18):
```

valeurs prises par k

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

La boucle for

Traduire en Python :

Pour un entier k allant de 3 à 17

La boucle for

Traduire en Python :

Pour un entier k allant de 3 à 17

réponse

```
for k in range(3,18):
```

valeurs prises par k

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

La boucle for

Traduire en Python :

Pour un entier k allant de 3 à 17 avec un pas de 2

La boucle for

Traduire en Python :

Pour un entier k allant de 3 à 17 avec un pas de 2

réponse

```
for k in range(3, 18, 2):
```

valeurs prises par k

3
5
7
9
11
13
15
17

La boucle for

Pour chaque ligne de code, donner les valeurs prises par i :

■ `for i in range(6):`

■ `for i in range(2,9):`

■ `for i in range(2,19,4):`

La boucle for

Pour chaque ligne de code, donner les valeurs prises par i :

■ **|** `for i in range(6):`

réponse

0 ; 1 ; 2 ; 3 ; 4 ; 5

■ **|** `for i in range(2,9):`

réponse

2 ; 3 ; 4 ; 5 ; 6 ; 7 ; 8

■ **|** `for i in range(2,19,4):`

réponse

2 ; 6 ; 10 ; 14 ; 18

La boucle for

Dans chaque cas, dire ce qui s'affiche dans la console :

```
| for k in range(3,10):  
    print(k)
```

```
| for k in range(3,10):  
    print("bonjour")
```

La boucle for

Dans chaque cas, dire ce qui s'affiche dans la console :

```
for k in range(3,10):  
    print(k)
```

```
for k in range(3,10):  
    print("bonjour")
```

réponse

dans la console

```
>>>
```

```
3
```

```
4
```

```
5
```

```
6
```

```
7
```

```
8
```

```
9
```

dans la console

```
>>>
```

```
bonjour
```

```
bonjour
```

```
bonjour
```

```
bonjour
```

```
bonjour
```

```
bonjour
```

```
bonjour
```

La boucle for

Dans chaque cas, dire ce qui s'affiche dans la console :

```
x=0
for k in range(3,10):
    x=x+1
    print(x)
```

```
x=0
for k in range(3,10):
    print(x)
    x=x+1
```

```
x=0
for k in range(3,10):
    x=x+1
    print(x)
```

La boucle for

Dans chaque cas, dire ce qui s'affiche dans la console :

```
x=0
for k in range(3,10):
    x=x+1
    print(x)
```

```
x=0
for k in range(3,10):
    print(x)
    x=x+1
```

```
x=0
for k in range(3,10):
    x=x+1
    print(x)
```

réponse

dans la console

```
>>>
1
2
3
4
5
6
7
```

dans la console

```
>>>
0
1
2
3
4
5
6
```

dans la console

```
>>>
7
```

Les boucles non bornées

Lorsque le nombre de répétitions n'est pas connu à l'avance, on utilise une **boucle non bornée**. Elle permet de répéter un bloc d'instruction tant qu'une condition est vérifiée.

La structure correspondant à une boucle non bornée est « **while condition** : »

Si la condition est toujours réalisée, la boucle se répétera indéfiniment. Il est donc important de vérifier que la condition cesse d'être vraie à terme afin que la boucle se termine.

Le nombre de passage dans une boucle non bornée étant inconnu au départ, il peut être nécessaire de créer une variable comptant le nombre de tours. Elle peut indiquer à partir de quand une condition n'est plus vérifiée.

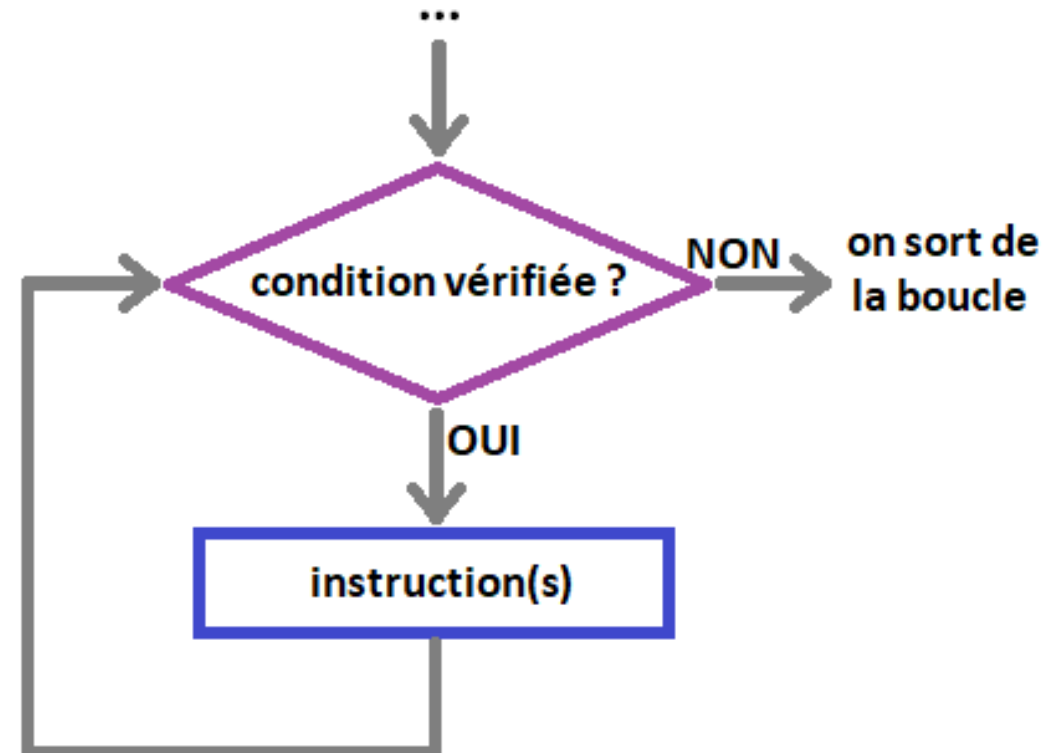
Une variable servant de **compteur** permet de compter le nombre de passages dans la boucle dans une boucle non bornée.

La boucle while

CE QU'IL FAUT SAVOIR

Python	syntaxe	pseudo-code
<pre>while x < 0 : x = x + 1</pre>	<pre>while condition : instruction(s)</pre>	Tant que $x < 0$ $x \leftarrow x + 1$

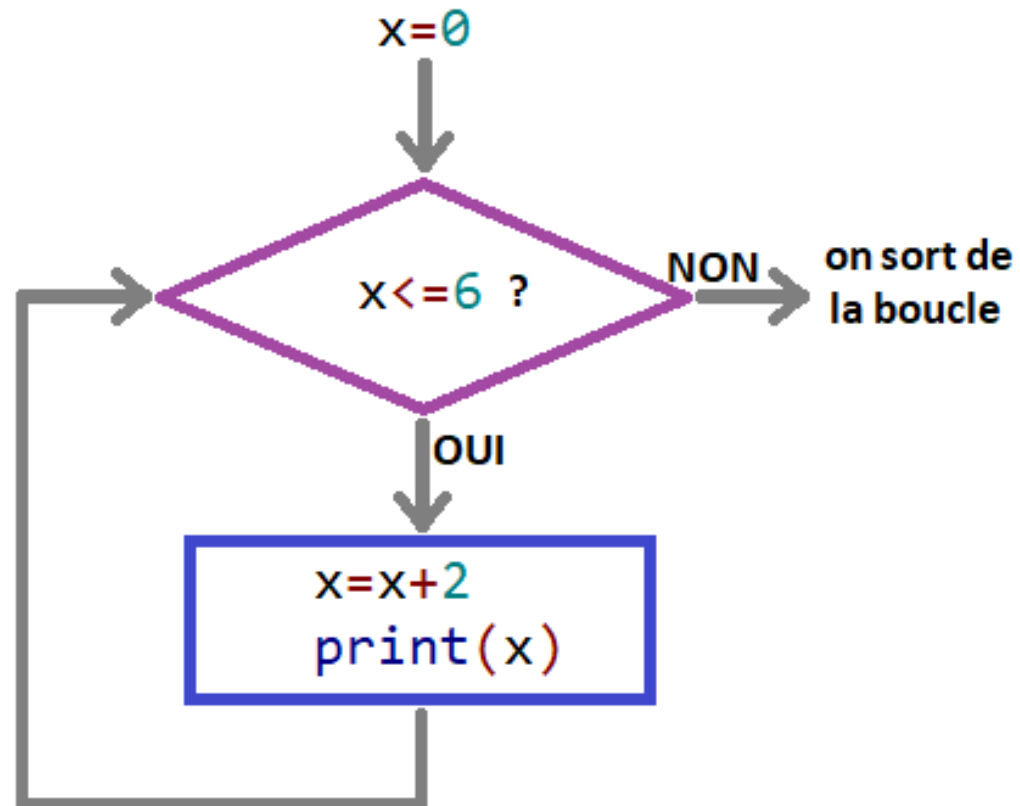
La boucle s'arrête quand la condition n'est plus vérifiée.



La boucle while

Exemple

```
x=0  
while x<=6 :  
    x=x+2  
    print(x)
```



dans la console

```
>>>  
2  
4  
6  
8
```

La boucle while

Dans chaque cas, dire ce qui s'affiche dans la console :

```
n=1
while n<1000 :
    n=n*20
    print(n)
```

```
x=5
n=1
while n<5 :
    x=x*20
    n=n+1
    print(x)
```

réponse

dans la console

```
>>>
20
400
8000
```

dans la console

```
>>>
100
2000
40000
800000
```

La boucle while

Que va-t-il se passer ?

```
x=0  
y=0  
while x<=8 :  
    y=x+1  
    print (y)
```

La boucle while

Que va-t-il se passer ?

```
x=0  
y=0  
while x<=8 :  
    y=x+1
```

réponse

La variable x va garder la valeur 0, et donc la condition $x \leq 8$ sera toujours vérifiée.

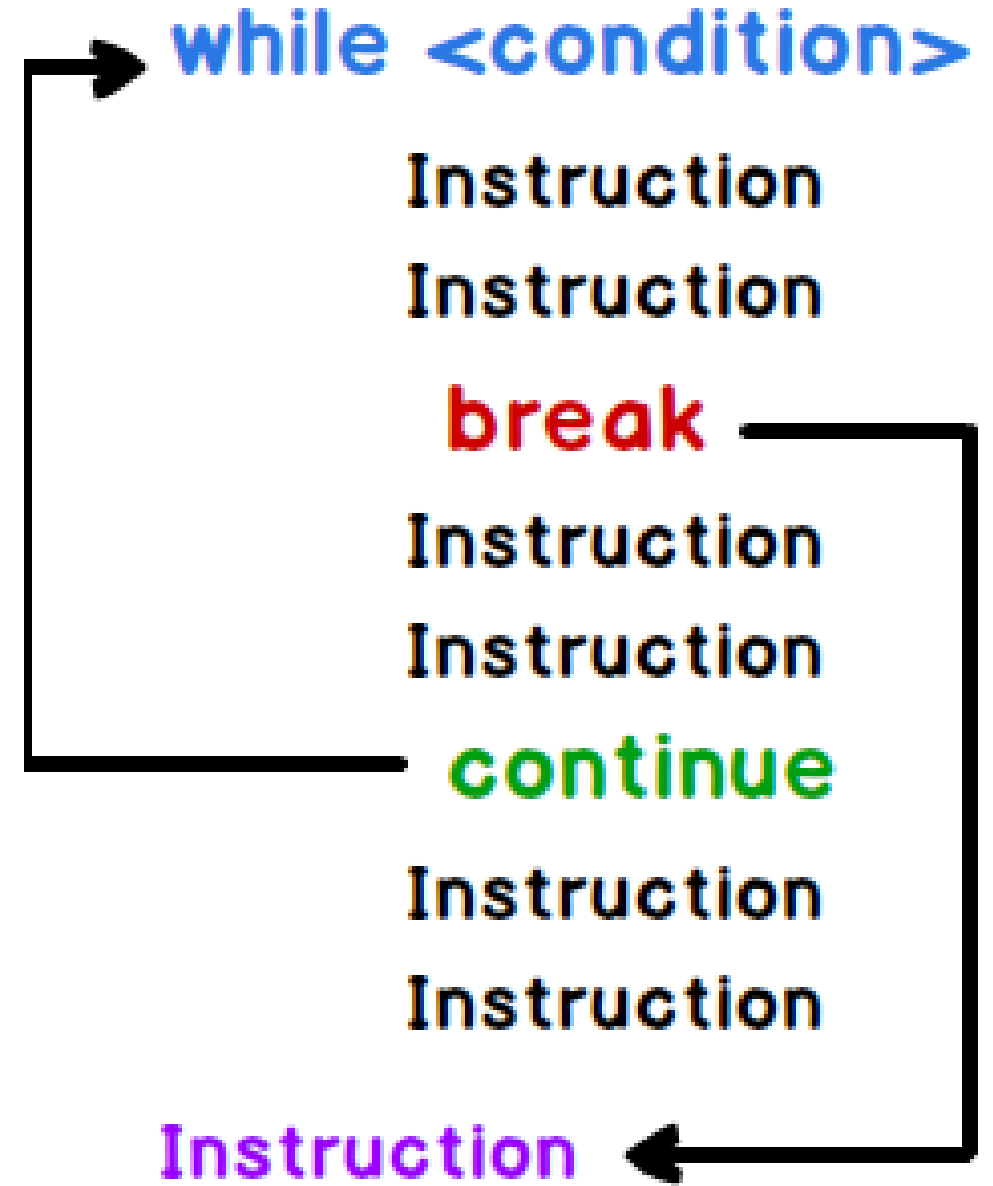
On a ce qu'on appelle « une boucle infinie », c'est-à-dire que rien ne permet de sortir de la boucle, il n'y a pas d'arrêt à l'algorithme (et donc ce ne sera pas un algorithme).

Mais dans ce cas, comment arrêter une boucle infinie avec EduPython ?

Les instructions break et continue

break est utilisé pour quitter une boucle while/for,

alors que **continue** est utilisé pour ignorer le bloc actuel et revenir à l'instruction while/for.



L'instruction break

Exemple avec While

Code

```
i = 0
while True:
    print(i)
    i = i + 1
    if i >= 4:
        break
```



Console

```
0
1
2
3
```

L'instruction break

Exemple avec for

Code

```
for val in "Python":  
    if val == "h":  
        break  
    print(val)
```

Console



P
y
t

L'instruction continue

Exemple avec While

Code

```
i = 10
while i > 0:
    i = i - 1
    if i == 5:
        continue
    print(i)
```



Console

9
8
7
6
4
3
2
1
0

L'instruction continue

Exemple avec for

Code

```
for val in "Python":  
    if val == "h":  
        continue  
    print(val)|
```



Console

P
y
t
o
n

Les Boucles imbriquées

Il est possible d'imbriquer des boucles. A chaque passage dans la première boucle, la deuxième boucle est effectuée entièrement.

Exemple : programme qui affiche les tables de multiplications de 7, 8 et 9. La première boucle fait varier i de 7 à 9 et pour chaque valeur de i, la variable j varie de 1 à 10 dans la deuxième boucle.

```
for i in range(7,10) :  
    print("table de ",i)  
    for j in range(1,11) :  
        print(i*j)
```